Enterprise Internal Assistant (Slack RAG Bot PoC)

Technical Description & Local Workstation Setup Guide

Project: Enterprise Internal Assistant (Slack RAG Bot)	**Course:** COS40005 - Swinburne University
Project Lead: Mr. Truong Ngoc Gia Hieu	**Client / Sponsor:** Mr. Phat (Eric) Pham
Project Period: Sep 29, 2026 – Dec 6, 2026	**Financial Constraint:** $0.00 Budget (Local PoC)

1. Overview of Local Workstation Setup

The **Enterprise Internal Assistant** is an AI-powered conversational Slack bot developed to eliminate knowledge fragmentation across corporate repositories (such as Confluence, technical documentation, and project repositories). Operating under the strict financial constraint of a **zero-dollar budget ($0.00)** during the development and Proof of Concept (PoC) phase, the solution utilizes a local workstation deployment model.

By executing Large Language Model (LLM) inference, text embedding generation, vector storage, and orchestration logic directly on local developer workstations, the system guarantees **strict enterprise data privacy** (preventing confidential documents from leaving local disk storage) while completely eliminating cloud infrastructure costs.

Component	Technology	Role & Architecture Justification
Runtime Environment	Python 3.10+	Core server runtime hosting RAG orchestration pipelines, event loops, and data ingestion.
AI Inference Engine	Ollama	Local execution of Llama-3 / Mistral LLMs and nomic-embed-text embedding models.
Vector Database	ChromaDB	Embedded, disk-persistent vector store for chunk embeddings and similarity retrieval.
Bot Messaging API	Slack Socket Mode	Bi-directional WebSocket connection. Bypasses public IPs, firewall rules, and Ngrok tunnels.
Access Control	Channel Whitelist	Enforces strict channel-level access boundaries alongside corporate Slack SSO.

2. Prerequisites & Core Software Requirements

Before initializing the workstation environment, verify that the local system meets the following specifications:

- **Python 3.10+:** Required runtime for LangChain / LlamaIndex orchestration and Slack Bolt framework execution.
- **Git Version Control:** Required for collaborative code maintenance and packaging source code for handover.
- **VS Code (or preferred IDE):** Environment for editing scripts, environment variables, and inspecting local logs.
- **System Hardware:** Minimum 16 GB RAM (32 GB recommended) and 10 GB free disk space for local LLM weights.

3. Local AI & Embedding Engine Setup (Ollama)

Ollama provides local LLM and embedding model execution on workstation hardware without sending data to third-party endpoints.

Step 3.1: Download and Install Ollama Engine

```
# Download and run installer from official Ollama release site
# (Follow standard desktop installer wizard for macOS / Windows / Linux)
```

Step 3.2: Pull Local Model Weights

```
# Pull Llama-3 8B quantized LLM for response synthesis:
ollama pull llama3

# Pull Nomic Text Embedding model for vector indexing:
ollama pull nomic-embed-text
```

Step 3.3: Test Local Inference Service

```
# Test REST endpoint service running locally on port 11434:
# Verify active model execution via CLI or API health check
```

4. Vector Database Initialization (ChromaDB)

ChromaDB operates in local persistent mode, saving collection data and document embeddings to disk at ./data/chroma_db.

```
# Persistent Vector DB Initialization in Python:
import chromadb
from chromadb.config import Settings

# Instantiate persistent disk-backed vector store
chroma_client = chromadb.PersistentClient(path='./data/chroma_db')
collection = chroma_client.get_or_create_collection(name='enterprise_knowledge')
```

5. Slack App & Socket Mode Configuration

To enable event-driven communication without public webhooks or Ngrok tunnels, the Slack App utilizes **Socket Mode** via WebSockets.

Required Token Architecture & Permissions:

1. **App-Level Token:** Enable Socket Mode in App Settings and assign the connections:write scope.
2. **Bot User OAuth Token:** Grant scopes: app_mentions:read, chat:write, channels:history, and im:history.
3. **Event Subscriptions:** Subscribe to app_mention and message.channels events.
4. **Channel Whitelisting Enforcement:** Incoming messages are validated against ALLOWED_CHANNEL_IDS before executing RAG retrieval.

6. Python Environment & Dependency Installation

Configure the isolated Python virtual environment and install project dependencies:

```
# Create and activate virtual environment
python3 -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate

# Upgrade pip and install core PoC dependencies
pip install --upgrade pip
pip install slack-bolt langchain langchain-community chromadb ollama python-dotenv
```

7. Environment Configuration (.env Template)

Store execution credentials and settings in a root .env file:

```
# ============================================================
# LOCAL WORKSTATION CONFIGURATION FILE (.env)
# ============================================================

# Slack Credentials (Socket Mode)
SLACK_BOT_TOKEN=<your_bot_user_oauth_token>
SLACK_APP_TOKEN=<your_app_level_token>

# Access Control Whitelist (Comma-separated Channel IDs)
ALLOWED_CHANNEL_IDS=C0123456789,C0987654321

# Local AI Engine Settings (Ollama)
OLLAMA_BASE_URL=http://localhost:11434
LLM_MODEL=llama3
EMBEDDING_MODEL=nomic-embed-text

# Local Vector Store Storage Path
CHROMA_PERSIST_DIR=./data/chroma_db
```

8. RAG Execution Workflow & Architecture Summary

Step	Stage	Technical Execution Details
1	User Mention	Employee prompts bot in whitelisted Slack channel (e.g., @Assistant how to setup VPN?).
2	Socket Event	Slack Bolt listener captures event over WebSocket using SLACK_APP_TOKEN.
3	Whitelist Check	Bot verifies channel_id against authorized ALLOWED_CHANNEL_IDS.
4	Vector Search	Query vector generated via Ollama and matched against persistent ChromaDB index.
5	LLM Synthesis	Matching document context + query passed to local Llama-3 for response synthesis.
6	Slack Reply	Grounded answer with reference source metadata posted back to Slack thread.